Intro to ML

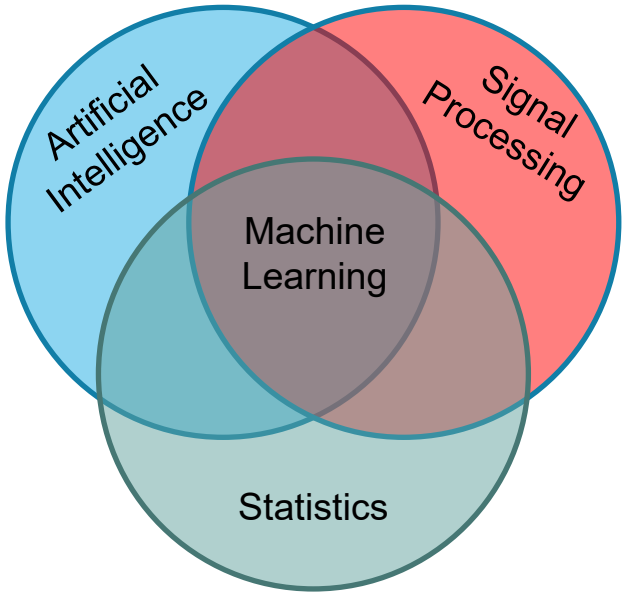
What is Machine Learning?

- A field focused on building models that learn patterns from data to make predictions or decisions
- Uses statistical modeling, optimization, and computation to improve performance without explicit rule-based programming
- Formal definition (Mitchell, 1997):
  - A program learns from experience E with respect to task T and performance measure P if its performance on T, as measured by P, improves with E.

BYU

Roots of machine learning

- Machine learning emerged at the intersection of artificial intelligence, statistics, signal processing, and optimization
- Examples of contributions:
  - **AI**: Representation learning, neural networks, search, and decision-making
  - **SP**: information theory, detection and estimation theory
  - **Stats**: regression, classification, a probabilistic framework for handling uncertainty and randomness in the data



BYU

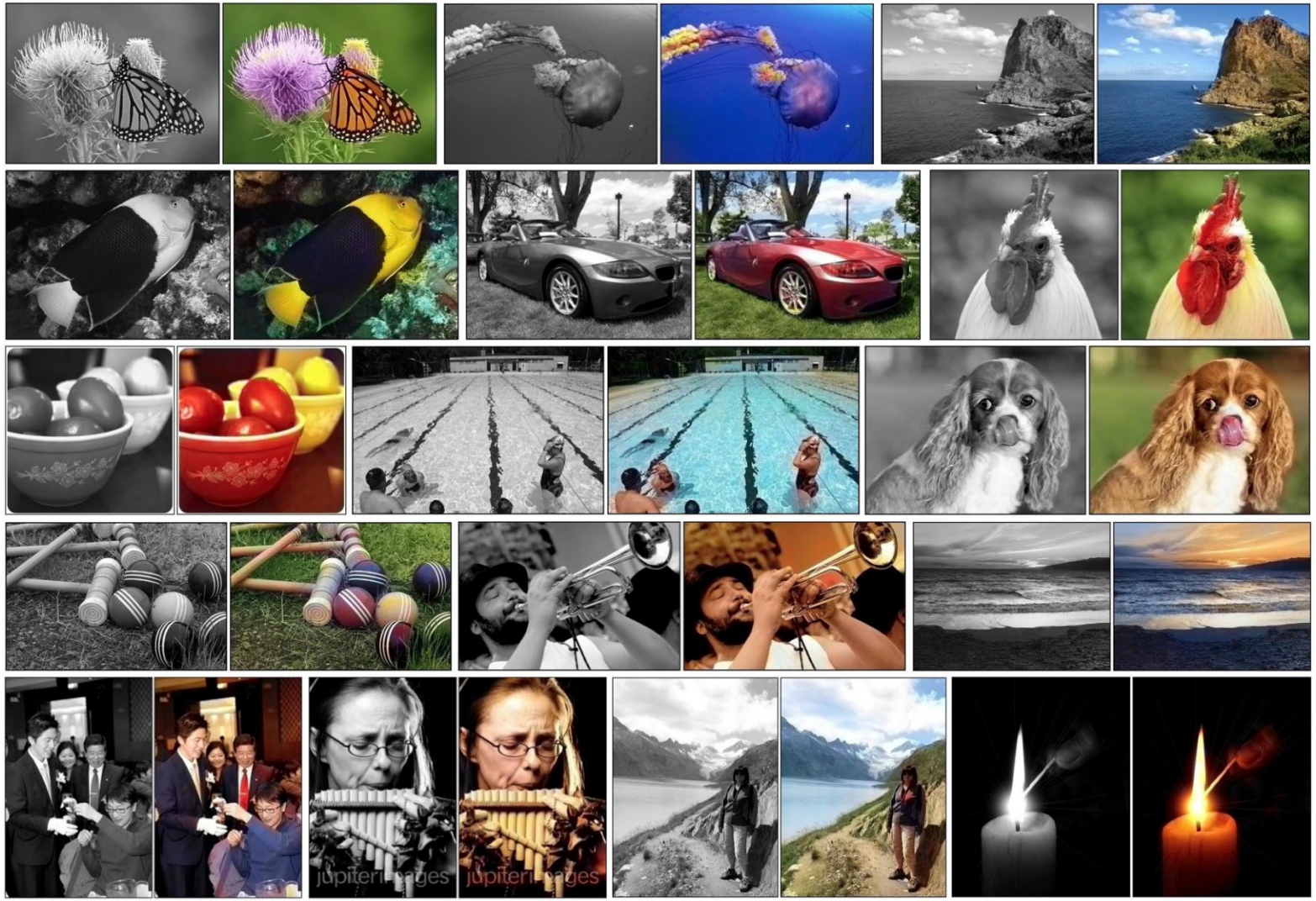
Classical Statistics vs ML

- What are the differences between classical statistics and ML?
  - There is an overlap of methods
- Somewhat broadly, classical statistics focuses heavily on data modeling
- ML typically focuses on the endgame/output (e.g., prediction)
  - A model is useless if it doesn't give a good prediction
- Many ML methods are based on statistical models
- See Breiman (2001) for one viewpoint on the two fields: "Statistical Modeling: The Two Cultures"

BYU

Success of Machine Learning/AI

Image colorization (Zhang et al., 2016)

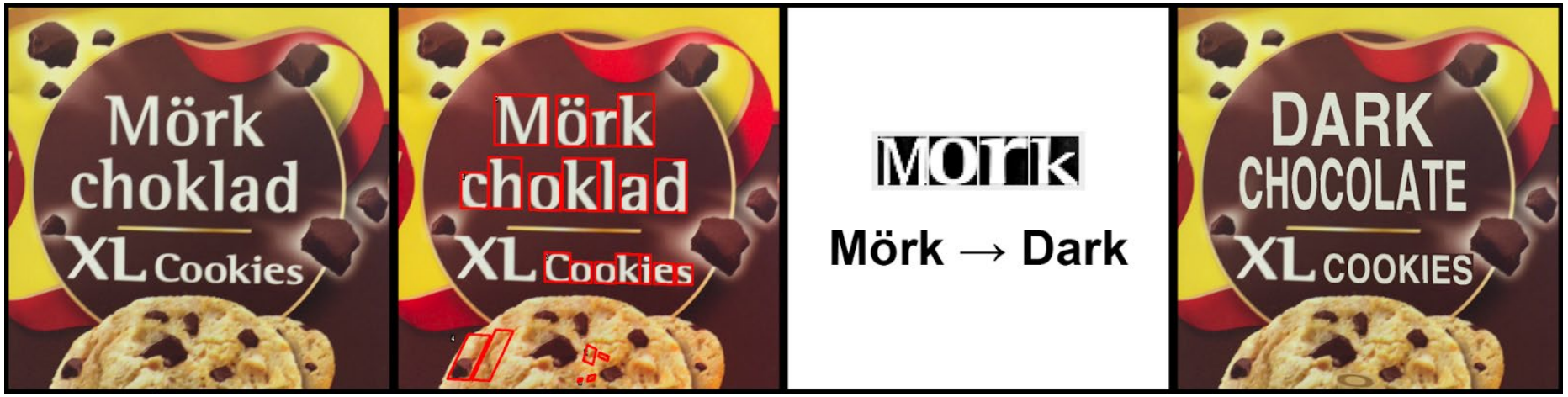


BYU

Success of Machine Learning/AI

Real-time visual translation on smartphones

1. Find the letters
2. Recognize the letters
3. Translate
4. Render the translation in the same style



Google blog, 2015

BYU

Success of Machine Learning

Automated programming with ChatGPT

Find the bug with this code:

```
for (var i = 0; i < 5; i++) {
  setTimeout(() => console.log(i), 1000)
}
```

The code contains a bug that will cause it to always print the number 5 to the console, rather than the expected sequence of numbers from 0 to 4. This is because the `setTimeout` function creates a closure that captures the value of the `i` variable, but the `i` variable is changed before the callback function is executed.

To fix this bug, the code can be modified to create a new variable in the loop that is passed to the `setTimeout` callback function, rather than using the loop variable directly. This will ensure that each callback function captures the correct value of the loop variable, and the expected output will be produced. Here is an example of how the code could be fixed:

Copy code

```
for (var i = 0; i < 5; i++) {
  // Create a new variable and pass its value to setTimeout
  const value = i;
  setTimeout(() => console.log(value), 1000)
}
```

This code will print the numbers 0 to 4 to the console, as expected. Alternatively, the `let` keyword can be used in place of `var` to declare the loop variable, which will automatically create a new variable for each iteration of the loop and avoid the need to create a new variable manually. This is a common pattern for avoiding closure-related bugs in JavaScript.

BYU

Success of Machine Learning

AI generated art

Success of Machine Learning

Automatic game playing

Alpha Go Zero

nature
International journal of science

Access provided by Yale University

Altmetric: 2188 Citations: 1 More detail »

Article

Mastering the game of Go without human knowledge

David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel & Demis Hassabis

Nature **550**, 354–359 (19 October 2017)
doi:10.1038/nature24270
Download Citation

Computational science
Computer science Reward

Received: 07 April 2017
Accepted: 13 September 2017
Published online: 18 October 2017

Alpha Zero

arXiv.org > cs > arXiv:1712.01815

Search or
(Help | Advanc

Computer Science > Artificial Intelligence

Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm

David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, Demis Hassabis

(Submitted on 5 Dec 2017)

The game of chess is the most widely-studied domain in the history of artificial intelligence. The strongest programs are based on a combination of sophisticated search techniques, domain-specific adaptations, and handcrafted evaluation functions that have been refined by human experts over several decades. In contrast, the AlphaGo Zero program recently achieved superhuman performance in the game of Go, by tabula rasa reinforcement learning from games of self-play. In this paper, we generalise this approach into a single AlphaZero algorithm that can achieve, tabula rasa, superhuman performance in many challenging domains. Starting from random play, and given no domain knowledge except the game rules, AlphaZero achieved within 24 hours a superhuman level of play in the games of chess and shogi (Japanese chess) as well as Go, and convincingly defeated a world-champion program in each case.

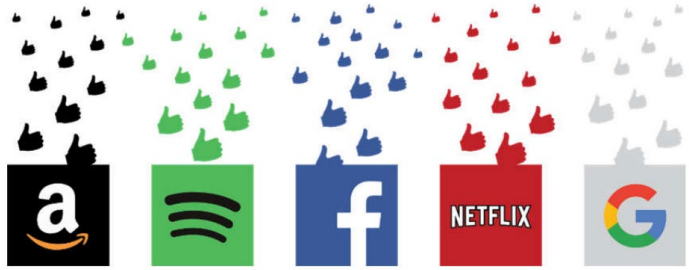
Subjects: **Artificial Intelligence (cs.AI)**; Learning (cs.LG)
Cite as: **arXiv:1712.01815 [cs.AI]**
(or **arXiv:1712.01815v1 [cs.AI]** for this version)

Submission history
From: David Silver [view email]
[v1] Tue, 5 Dec 2017 18:45:38 GMT (272kb,D)

BYU

Other Machine Learning Applications

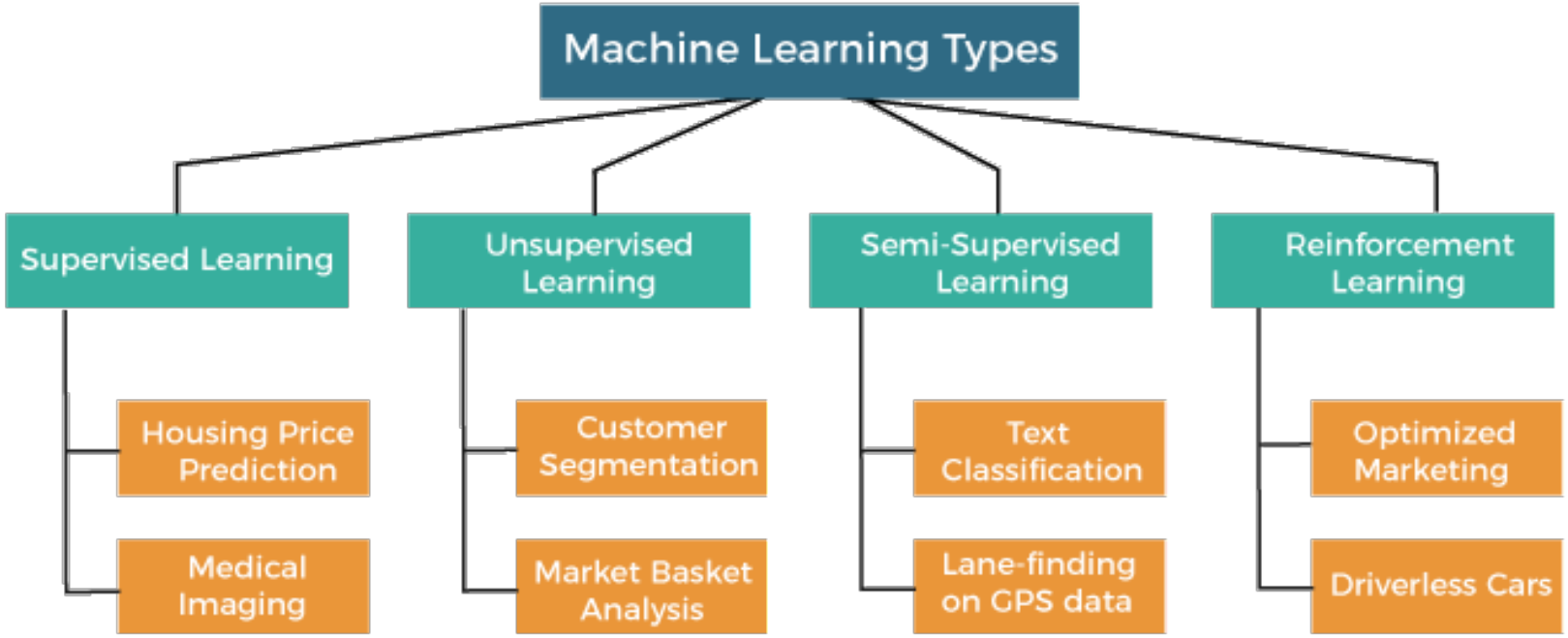
- Chatbots (NLP)
- Fraud detection
- Route planning
- Search engine refinement
- Face recognition/detection
- Credit risk assessment
- Financial market prediction
- Medical diagnosis
- Personalized medicine
- Electricity demand forecasting
- Spam filtering
- Collision avoidance systems
- Speech synthesis/analysis (e.g. Siri, Alexa)
- Recommender systems (e.g. Netflix, Spotify)



BYU

MACHINE LEARNING TYPES

- **Supervised**
  - Classification
  - Regression
- **Unsupervised**
  - Clustering
  - Dimensionality Reduction
- **Semi-supervised**
- **Reinforcement**



BYU

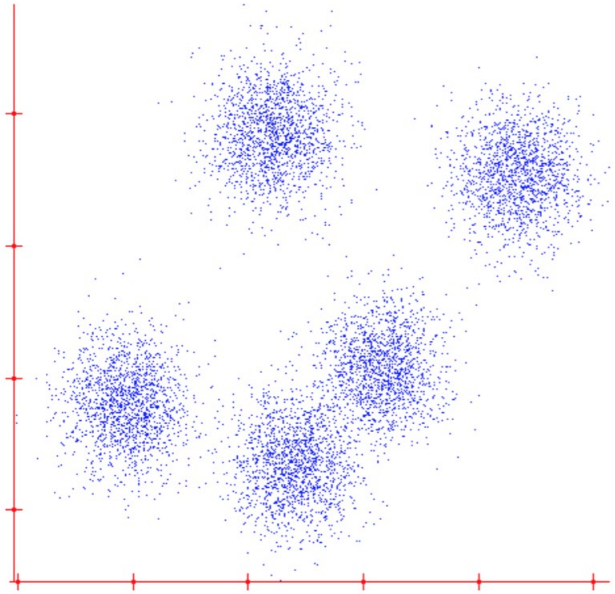
QUIZ

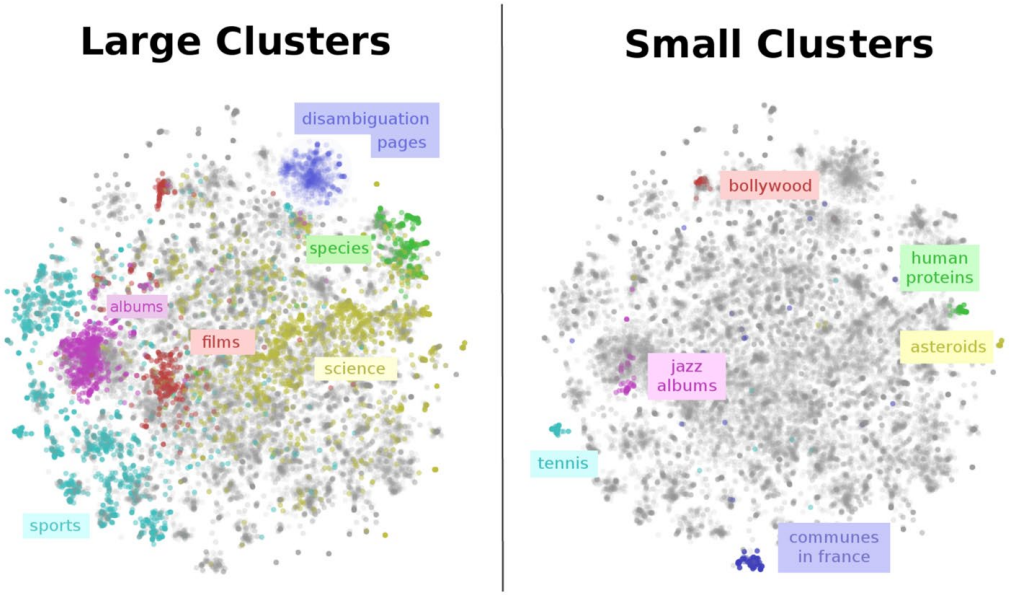
Of the following examples, which would you address using an <u>unsupervised</u> learning algorithm? (Check all that apply.)

- [ ] Given email labeled as spam/not spam, learn a spam filter.
- [x] Given a set of news articles found on the web, group them into set of articles about the same story.
- [x] Given a database of customer data, automatically discover market segments and group customers into different market segments.
- [ ] Given a dataset of patients diagnosed as either having diabetes or not, learn to classify new patients as having diabetes or not.

BYU

CLUSTERING DATA: GROUP SIMILAR THINGS





wikipedia articles

[Goldberger et al.]

APPLICATIONS OF CLUSTER ANALYSIS

- **Customer Segmentation**: In marketing, clustering helps to group customers based on behavior, preferences, or demographics.
  - This segmentation enables personalized marketing, targeted promotions, and better customer service.

- **Document Clustering**: often used in natural language processing to organize large text corpora.
  - It groups similar documents, which is useful for topic modeling, or summarizing news articles.

- **Anomaly Detection**: Can identify outliers in data.
  - Ex: in network security, clustering can detect unusual patterns of behavior that may indicate a cyberattack or fraud.

BYU

MORE APPLICATIONS OF CLUSTER ANALYSIS

- **Social Network Analysis**: Clustering helps to find communities within social networks
  - by grouping users with similar connections, interests, or behaviors.
- **Genomics**: clustering is used to classify genes with similar expression patterns,
  - helping in the discovery of gene functions and understanding of diseases.
- **Urban Planning and Geographic Analysis**: Helps in analyzing geographic data
  - Ex: identifying areas with similar land use, demographic profiles, or crime rates, which aids in urban planning.

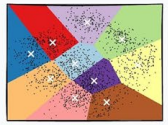
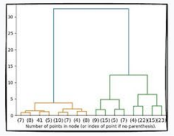
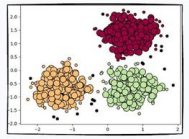
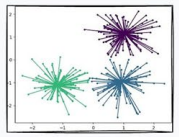
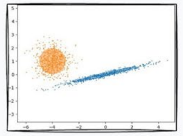
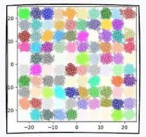
BYU

MAJOR TYPES OF CLUSTERING ALGORITHMS

Our focus:

6 Types of Clustering Algorithms in Machine Learning

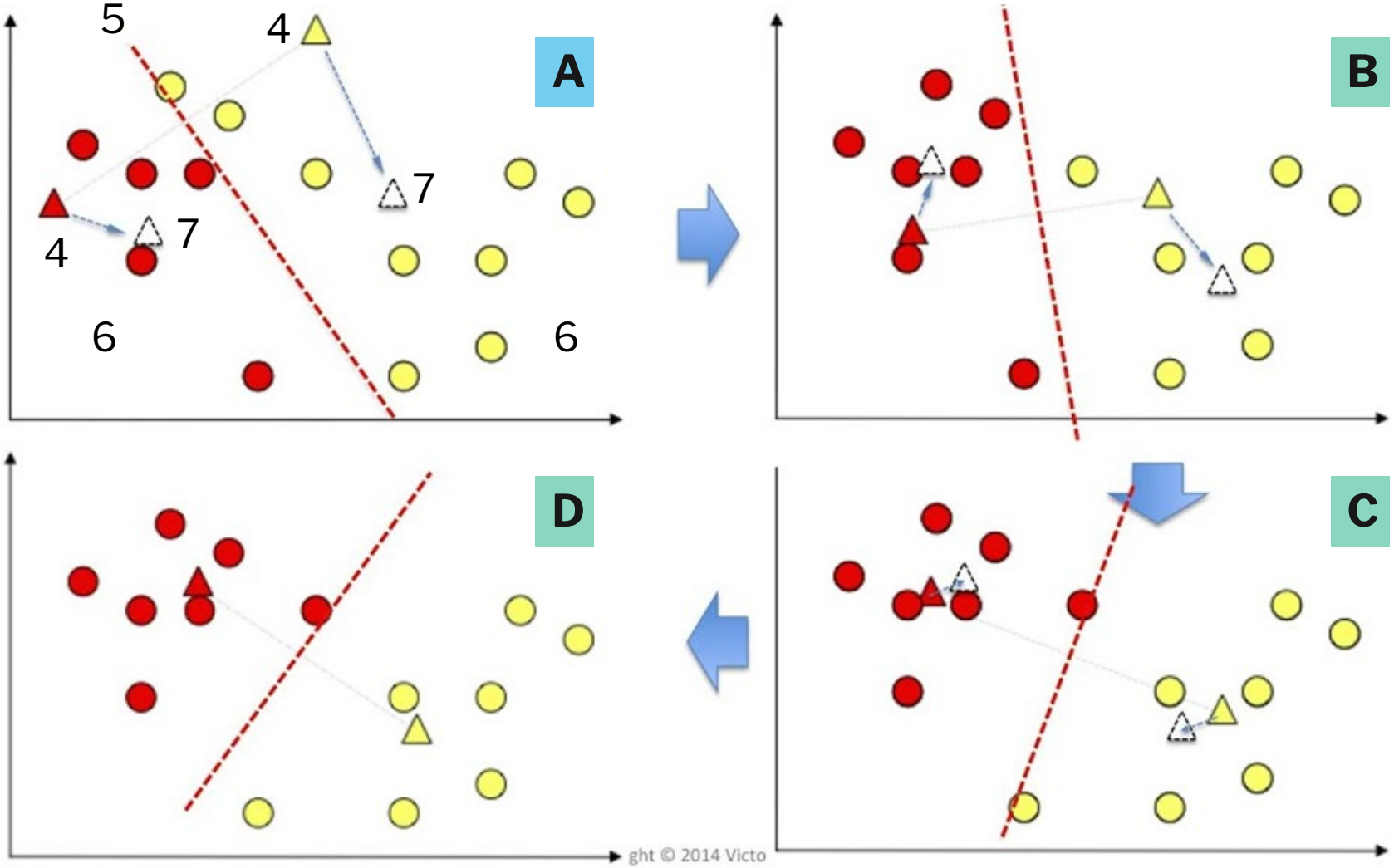
blog.DailyDoseofDS.com

Clustering Algorithm Type		Clustering Methodology	Algorithm(s)
	Centroid-based	Cluster points based on proximity to centroid	KMeans KMeans++ KMedoids
	Connectivity-based	Cluster points based on proximity between clusters	Hierarchical Clustering (Agglomerative and Divisive)
	Density-based	Cluster points based on their density instead of proximity	DBSCAN OPTICS HDBSCAN
	Graph-based	Cluster points based on graph distance	Affinitiy Propagation Spectral Clustering
	Distribution-based	Cluster points based on their likelihood of belonging to the same distribution.	Gaussian Mixture Models (GMMs)
	Compression-based	Transform data to a lower dimensional space and then perform clustering	BIRCH

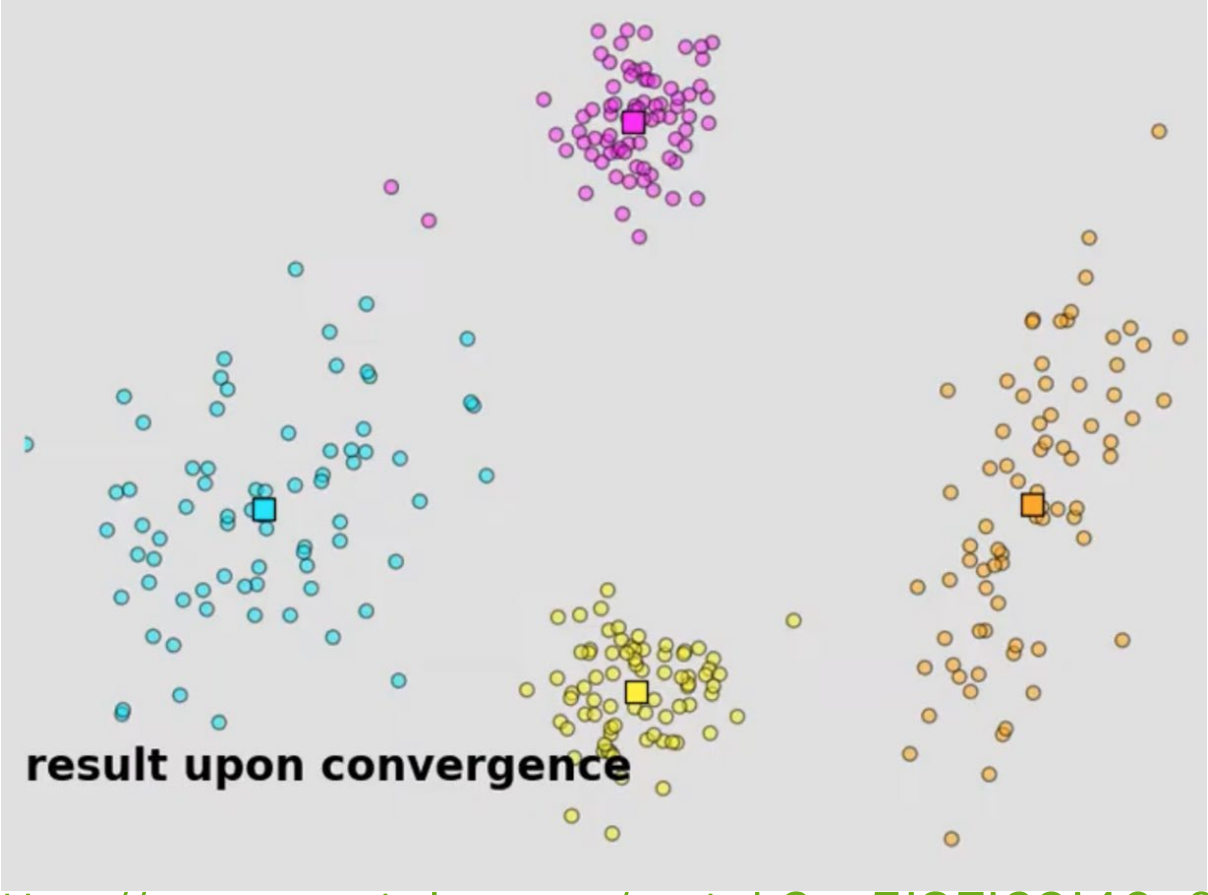
1. **Centroid -based:** Cluster data points based on proximity to centroids.
2. **Connectivity -based:** Cluster points based on proximity between clusters.
3. **Density -based:** Cluster points based on their density. It is more robust to clusters with varying densities and shapes than centroid -based clustering.
4. **Graph -based:** Cluster points based on graph distance.
5. **Distribution -based:** Cluster points based on their likelihood of belonging to the same distribution. Gaussian Mixture Model in one example.
6. **Compression -based:** Transform data to a lower dimensional space and then perform clustering

K-MEANS CLUSTERING

1. Standardize samples
2. Choose # clusters (K)
3. Choose # iterations
4. Randomly place initial centroids (Fig. A)
5. Calculate distance to centroid (Fig. A)
6. Assign samples with the nearest centroid color (Fig. A)
7. Move centroids to the mean of cluster (Fig. A)
8. Repeat 5-7 (Figs. B, C, D) until convergence



K-MEANS CLUSTERING ANIMATION



https://www.youtube.com/watch?v=5I3Ei69I40s&t=59s

K-MEANS DETAILS

- For a very good detailed explanation of how K-Means works under the hood:

  https://www.youtube.com/watch?v=IX-3nGHDhQg

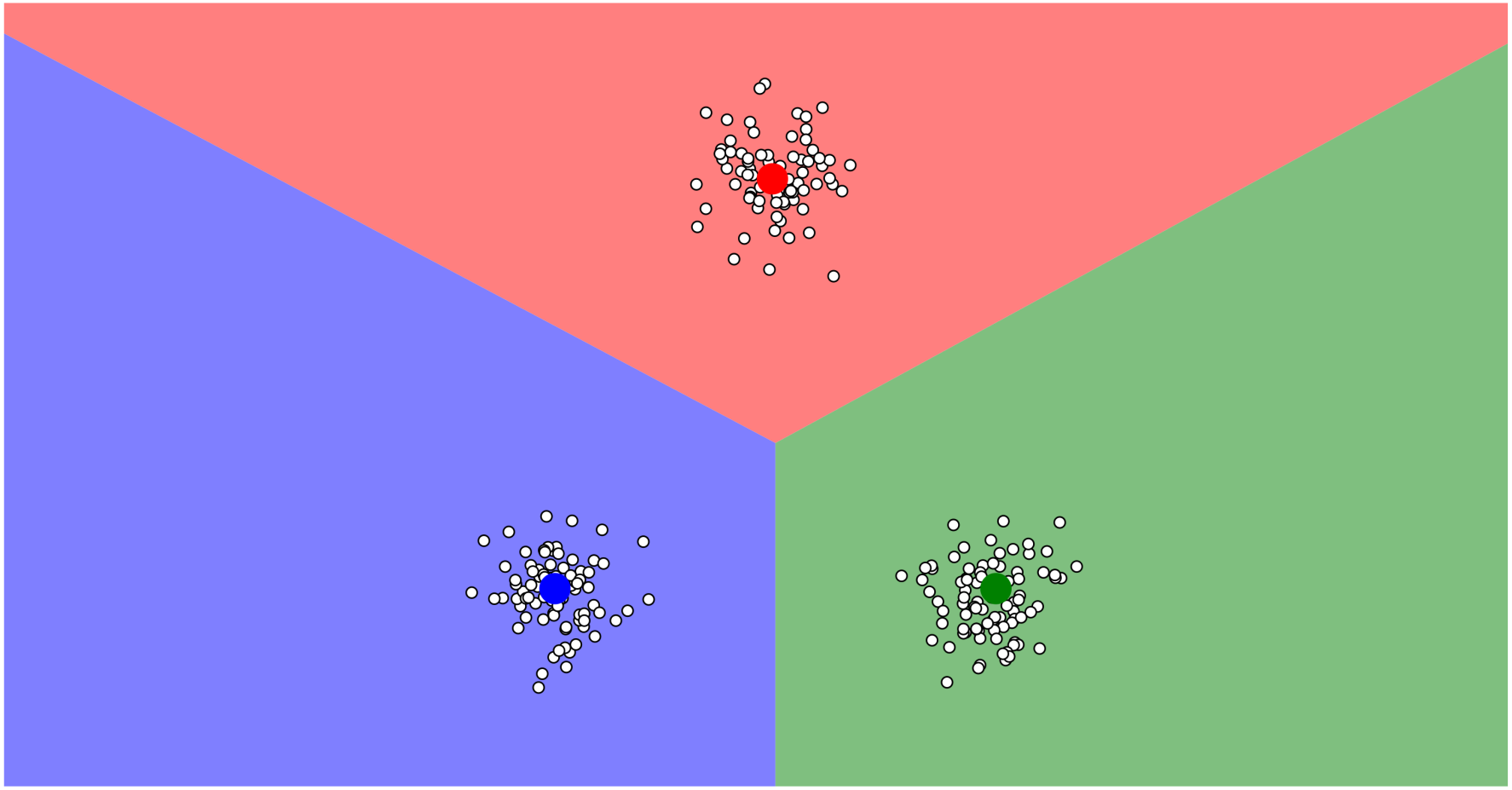
- Another very good video, if you can understand his accent, is:
  https://www.youtube.com/watch?v=iNIZ3IU5Ffw&t=300s

- What are some of the advantages/disadvantages of the K-Means algorithm for clustering? When does it work well? When does it fail?

BYU

Visualizing KMeans

https://www.naftaliharris.com/blog/visualizing-k-means-clustering/



EVALUATION METHOD: WITHIN CLUSTER SUM OF SQUARES (WCSS

- Measures how compact the clusters are
- For each cluster, compute the squared distance from each point to its centroid
- Sum these distances across all clusters

$$\text{WCSS} = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

Interpretation:

- Smaller WCSS → tighter clusters
- Used in the *elbow method* to choose K

BYU

WITHINCLUSTER SUM OF SQUARES (WCSS) CODE

```python
import numpy as np

def calculate_wcss(data, labels, centroids):
    wcss = 0
    for i, centroid in enumerate(centroids):
        # Get points in cluster i
        cluster_points = data[labels == i]
        # Calculate the sum of squared distances to the centroid
        wcss += np.sum((cluster_points - centroid) ** 2)
    return wcss
```

In this example:

- `data` is your dataset as a NumPy array,
- `labels` are the cluster labels for each data point (indicating cluster assignment),
- `centroids` are the coordinates of each cluster centroid.

BYU

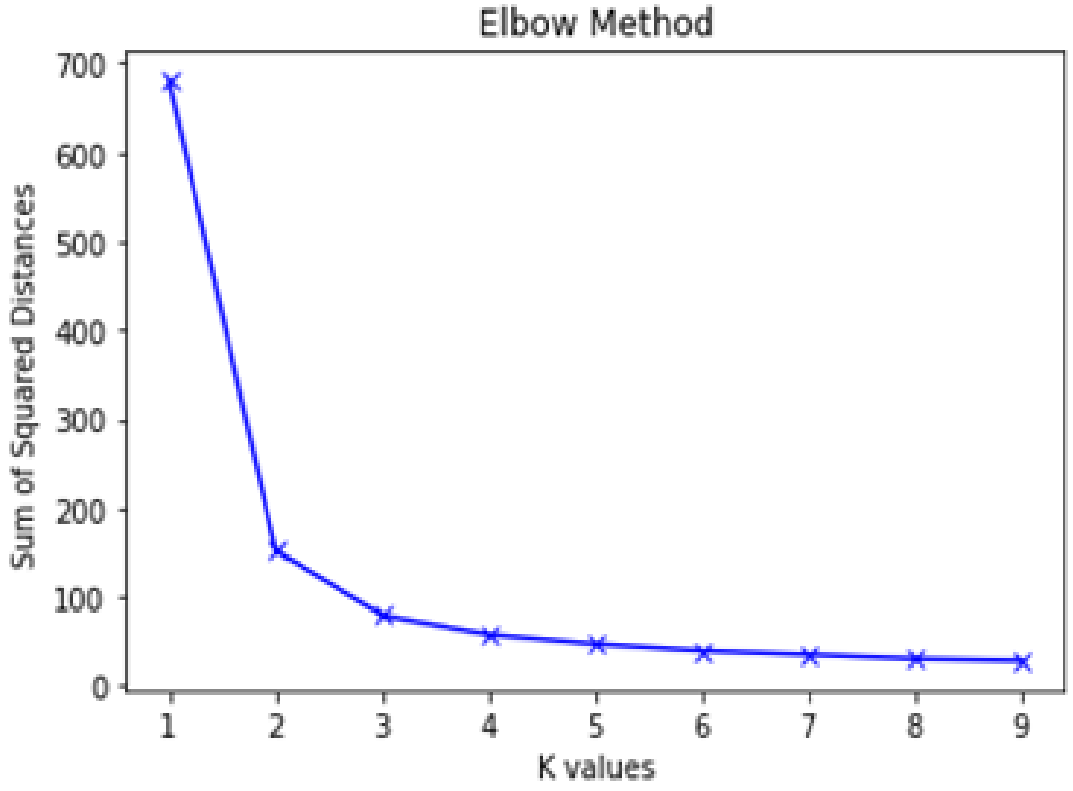
FINDING THE OPTIMAL K VALUE USING ELBOW METHOD

Steps:

- Run k-means for different values of K.
- Compute WCSS for each K.
- Plot WCSS vs. K.
- Look for the “elbow” — where improvements begin to slow down.

Idea:

- Choose the KK at which WCSS reductions begin to show diminishing returns.



ELBOW PLOT CODE EXAMPLE

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

# Generate sample data
X, _ = make_blobs(n_samples=300, centers=4, cluster_std=1.0, ran

# List to store WCSS values for each k
wcss = []

# Run k-means with k values from 1 to 10 and calculate WCSS
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)  # inertia_ is the WCSS for the

# Plot the WCSS values for each k to find the "elbow" point
plt.figure(figsize=(8, 5))
plt.plot(range(1, 11), wcss, marker='o')
plt.title('Elbow Method for Optimal k')
plt.xlabel('Number of clusters (k)')
plt.ylabel('Within-Cluster Sum of Squares (WCSS)')
plt.show()
```

Silhouette Method

The silhouette method evaluates how well a clustering fits the data.

For each point, it asks:

- Is this point closer to its own cluster or to a neighboring cluster?

Silhouette score ranges from −1 to +1

- Close to **+1** → well-clustered
- Near **0** → between clusters
- Negative → possibly mis-labeled

Silhouette Method Intuition: Cohesion vs. Separation

- **Cohesion:**

How close is it to other points in its cluster?

- **Separation:**

How far is it from points in the nearest other cluster?

The silhouette score compares these two quantities.

Silhouette Method Formulation

Silhouette Score for a Point:

$$\text{score} = \frac{\text{separation} - \text{cohesion}}{\max(\text{cohesion}, \text{separation})}$$

Where:

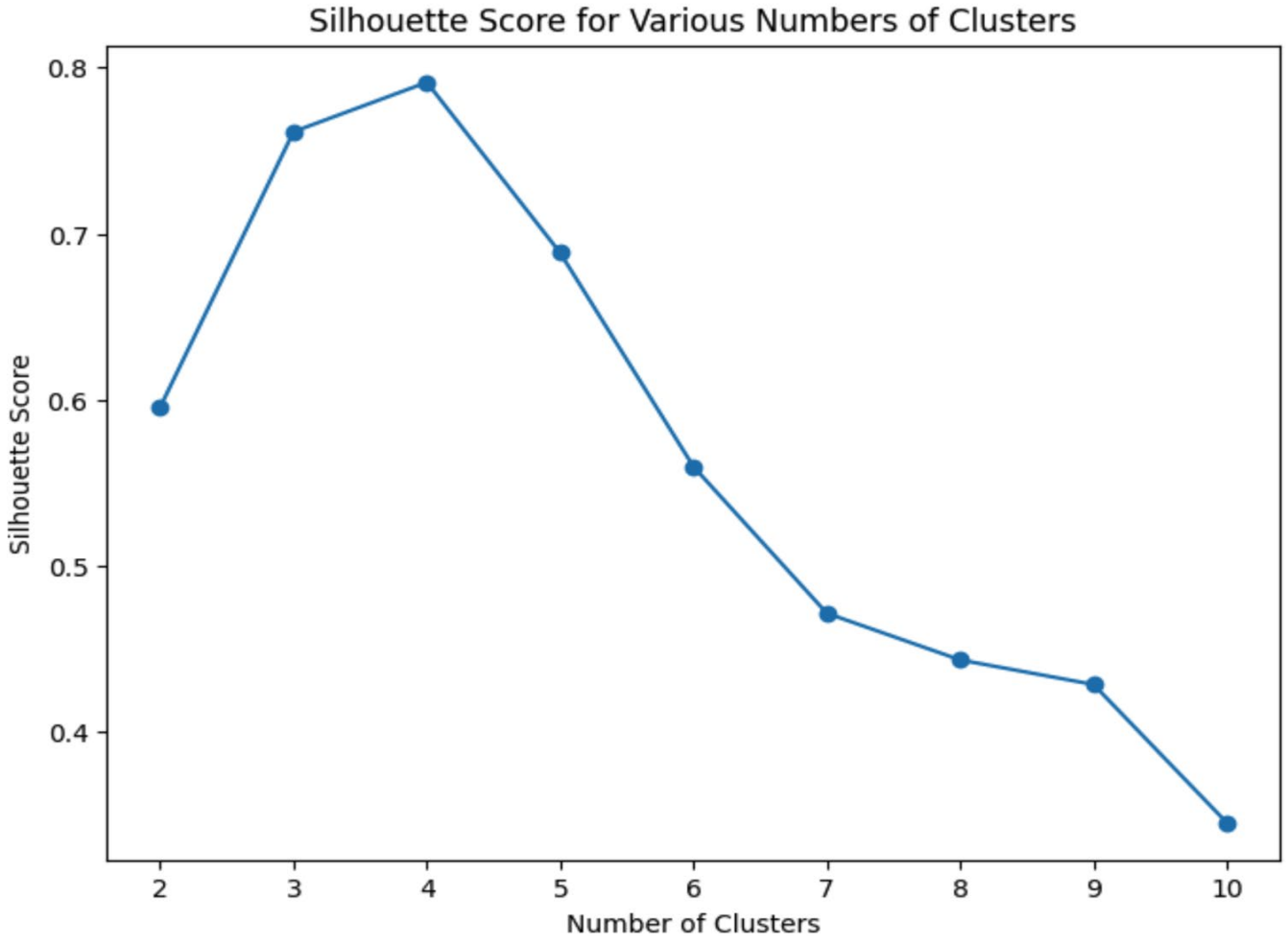
- **Cohesion:** average distance to points in its cluster
- **Separation:** average distance to points in the next nearest cluster
- **Overall silhouette score**: average across all points.

K Selection:

- Run clustering for different values of K.
- Compute the average silhouette score for each.
- Choose the K with the highest average score.

BYU

SILHOUETTE METHOD EXAMPLE



WSCC VS. SILHOUETTE

When to Use Each?

Metric	Measures	Best For	Limitations
WCSS	Intra-cluster compactness	Elbow Method, K-Means	Does not consider inter-cluster separation
Silhouette Score	Compactness + Separation	Comparing cluster quality across different algorithms	Computationally expensive

BYU